

CASO 2 – DESARROLLO GUIADO POR PRUEBAS (TDD)

Calculadora de edad · Calidad y Pruebas de Software · VIII ciclo

Integrantes	
Sección / Grupo	
Lenguaje y framework	
Enlace al repositorio	
Fecha	

1. Logro de la actividad

Al finalizar el caso, el estudiante desarrolla una funcionalidad aplicando el ciclo Red – Green – Refactor, escribiendo cada prueba unitaria antes del código, cubriendo casos normales, casos límite y casos de error, y evidenciando el proceso mediante su historial de commits.

Objetivos específicos:

1. Aplicar el ciclo Red – Green – Refactor de forma disciplinada, un requisito a la vez.
2. Diseñar pruebas unitarias para cálculos, validaciones y excepciones.
3. Identificar y probar casos límite (cumpleaños el mismo día, fechas futuras, años bisiestos).
4. Diseñar código testeable, evitando la dependencia directa de la fecha del sistema.
5. Refactorizar con seguridad, usando las pruebas como red de protección.

2. Recordatorio: ¿cómo funciona TDD?

Fase	Qué haces
● Red	Escribes una prueba pequeña para un comportamiento que aún no existe. La ejecutas y confirmas que falla (si pasa sin haber escrito código, la prueba no está probando nada nuevo).
● Green	Escribes el código mínimo para que esa prueba pase. No agregues funcionalidad que ninguna prueba exija todavía.
● Refactor	Mejoras el código (nombres, duplicación, estructura) sin cambiar su comportamiento. Todas las pruebas deben seguir pasando.

Regla de oro: no escribas código de producción si no hay una prueba fallando que lo pida. Repite el ciclo hasta cubrir todos los requisitos.

3. Descripción del caso

Una institución necesita una función que, a partir de la fecha de nacimiento de una persona, devuelva su edad exacta en años cumplidos a una fecha dada.

Requisitos funcionales

ID	Requisito	Detalle
RF1	Calcular la edad	Devuelve los años cumplidos entre la fecha de nacimiento y la fecha actual.
RF2	Cumpleaños pendiente	Si en el año actual la persona aún no cumple años, la edad es un año menos que la diferencia de años. Si cumple años hoy, ya cuenta el año.
RF3	Fecha no válida	Si la fecha de nacimiento es posterior a la fecha actual, se lanza una excepción con el mensaje: «La fecha de nacimiento no puede ser posterior a la fecha actual».
RF4	Años bisiestos	Una persona nacida el 29 de febrero cumple años el 29/02 en años bisiestos; en años no bisiestos se considera que cumple el 1 de marzo.

Decisión de diseño obligatoria: la fecha actual como parámetro

Si la función usa internamente la fecha del sistema (`date.today()`, `LocalDate.now()`, `DateTime.Now`), el resultado de la prueba cambia cada día y la prueba dejará de pasar con el tiempo. Por eso, la función debe recibir la fecha actual como parámetro:

```
calcular_edad(fecha_nacimiento, fecha_actual) -> int
```

En producción se llama con la fecha del sistema; en las pruebas se usa una fecha fija. Así cada prueba es repetible y determinista.

4. Preparación del entorno

Elige uno de estos lenguajes. Los ejemplos de esta guía están en Python, pero la lógica es la misma en todos.

Lenguaje	Framework de pruebas	Comando para ejecutar
Python 3	pytest	pytest -v
Java 17+	JUnit 5 (Maven o Gradle)	mvn test
C# (.NET)	xUnit o NUnit	dotnet test
JavaScript	Jest	npm test

1. Crea un repositorio en GitHub con el nombre `tdd-calculadora-edad-grupoX`.
2. Separa el código de producción del código de pruebas (por ejemplo: `src/edad.py` y `tests/test_edad.py`).
3. Haz un commit al final de cada fase con un prefijo que la identifique: `red: ...`, `green: ...`, `refactor: ...`. El historial de commits es la evidencia principal de que aplicaste TDD.

4. Toma una captura de pantalla de la ejecución de pruebas en cada fase Red (prueba fallando) y Green (pruebas pasando).

5. Desarrollo: ciclos TDD

Desarrolla los ciclos en orden. No adelantes código de un ciclo posterior.

Ciclo 1 (ejemplo resuelto): edad básica

- **Red** – Se escribe la prueba. Al ejecutarla falla porque la función `calcular_edad` no existe:

```
from datetime import date
from src.edad import calcular_edad

def test_edad_cuando_ya_cumplio_anios_este_anio():
    nacimiento = date(2000, 5, 15)
    hoy = date(2026, 9, 30)
    assert calcular_edad(nacimiento, hoy) == 26
```

- **Green** – Código mínimo para que pase (solo la diferencia de años; nada más):

```
def calcular_edad(fecha_nacimiento, fecha_actual):
    return fecha_actual.year - fecha_nacimiento.year
```

- **Refactor** – Por ahora el código es simple; no hay nada que mejorar. Commit: refactor: sin cambios en ciclo 1 (opcional).

Observa que este código es incompleto a propósito: falla con personas que aún no cumplen años este año. El siguiente ciclo lo pondrá en evidencia. Así trabaja TDD: cada prueba nueva obliga a mejorar el código.

Ciclo 2: cumpleaños aún no llega (RF2)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. Con el código del ciclo 1 devolverá 26 en lugar de 25.
Nombre de la prueba	<code>test_edad_cuando_aun_no_cumple_anios_este_anio</code>
Datos de entrada	Nacimiento: 10/12/2000 Fecha actual: 30/09/2026
Resultado esperado	Edad = 25
● Green	Ajusta el cálculo para restar 1 si el mes y día actuales son anteriores al mes y día de nacimiento.
● Refactor	Si ves duplicación o nombres poco claros, mejóralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

Prueba adicional del mismo ciclo: `test_edad_el_dia_del_cumpleanios` → nacimiento 30/09/2000, fecha actual 30/09/2026 → edad = **26**. Verifica que el día del cumpleaños ya cuenta el año.

Ciclo 3: fecha de nacimiento futura (RF3)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. Con el código actual devolverá un número negativo en lugar de lanzar la excepción.
Nombre de la prueba	test_fecha_nacimiento_futura_lanza_excepcion
Datos de entrada	Nacimiento: 01/01/2027 Fecha actual: 30/09/2026
Resultado esperado	Se lanza una excepción (ValueError / IllegalArgumentException / ArgumentException) con el mensaje «La fecha de nacimiento no puede ser posterior a la fecha actual».
● Green	Agrega al inicio de la función la validación que lance la excepción.
● Refactor	Si ves duplicación o nombres poco claros, méjoralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

```
# Pista en pytest para probar excepciones:
import pytest
with pytest.raises(ValueError, match="posterior a la fecha actual"):
    calcular_edad(date(2027, 1, 1), date(2026, 9, 30))
```

Ciclo 4: nacidos el 29 de febrero (RF4)

Elemento	Detalle
● Red	Escribe primero esta prueba y ejecútala. Debe fallar. Analiza si falla. Si pasa sin cambios, explica por qué en tu bitácora (ver nota).
Nombre de la prueba	test_bisiesto_un_dia_antes_del_cumpleanios test_bisiesto_cumple_el_1_de_marzo test_bisiesto_en_anio_bisiesto
Datos de entrada	Nacimiento: 29/02/2004 Fechas actuales: 28/02/2025 · 01/03/2025 · 29/02/2028
Resultado esperado	Edades: 20 · 21 · 24
● Green	Si alguna prueba falla, corrige el cálculo para respetar la regla del RF4.
● Refactor	Si ves duplicación o nombres poco claros, méjoralos. Vuelve a ejecutar todas las pruebas: deben seguir en verde.

Para pensar: si tu implementación compara (mes, día), es posible que estas pruebas pasen sin tocar el código. En TDD eso también es información: significa que el diseño ya cubría el caso. Lo importante es que **ahora existe una prueba que lo garantiza**. Si en cambio calculaste la edad con $\text{días} \div 365$, verás fallar estas pruebas.

Ciclo 5: refactorización final

Con todas las pruebas en verde, revisa el código completo y aplica al menos **dos** mejoras.

Ejemplos:

- Extraer la validación de la fecha a una función propia (validar_fecha_nacimiento).
- Extraer la comparación de mes y día a una función con nombre claro (ya_cumplio_anios).
- Mejorar nombres de variables y del mensaje de error (usar una constante).
- Eliminar código duplicado en las pruebas (fixtures o métodos de configuración).

Después de cada cambio ejecuta **todas** las pruebas. Si alguna falla, el refactor cambió el comportamiento: revierte y vuelve a intentarlo.

6. Bitácora de ciclos (completar)

Ciclo	Prueba escrita	Red: ¿por qué falló?	Green: ¿qué cambiaste?	Refactor realizado
1	test_edad_cuando_ya_cumplio_anios_esto_anio	La función no existía	Se creó calcular_edad con la resta de años	Ninguno
2				
3				
4				
5	—	—	—	

7. Preguntas de reflexión

1. ¿Por qué la función recibe la fecha actual como parámetro en lugar de obtenerla dentro? ¿Qué pasaría con tus pruebas dentro de un año si no fuera así?
2. En el ciclo 1 el código era incorrecto y la prueba pasaba. ¿Qué te enseña esto sobre la cantidad y variedad de pruebas necesarias?
3. ¿Las pruebas del ciclo 4 fallaron? Explica por qué sí o por qué no.
4. Relaciona este caso con las técnicas de la Semana 08: ¿qué clases de equivalencia y valores límite identificas para la fecha de nacimiento?
5. Menciona una ventaja y una desventaja de TDD frente a escribir las pruebas después del código.

8. Entregables

- Enlace al repositorio con el código, las pruebas y el historial de commits por fase (red / green / refactor).
- Este documento completado: datos del grupo, bitácora, capturas de cada fase Red y Green, y respuestas de reflexión.
- Captura final con todas las pruebas pasando (mínimo 7 pruebas).

9. Rúbrica de evaluación

Criterio	Puntos	Se obtiene el puntaje completo si...
Pruebas unitarias	5	Las 7 pruebas mínimas existen, tienen nombres descriptivos, usan fechas fijas y cubren RF1 a RF4.
Evidencia del ciclo TDD	5	El historial de commits muestra red → green → refactor en cada ciclo, con capturas de prueba fallando y pasando.

Criterio	Puntos	Se obtiene el puntaje completo si...
Código funcional	3	Todas las pruebas pasan y la función cumple los cuatro requisitos.
Refactorización	3	Se aplican al menos dos mejoras sin romper las pruebas, y se explican en la bitácora.
Bitácora y reflexión	4	La bitácora está completa y las respuestas demuestran comprensión de TDD y su relación con el diseño de casos de prueba.
Total	20	